Deep Learning for Economics and Finance

Azinovic–Yang–Zemlicka: shock-history inputs for high-dimensional models

Simon Scheidegger

HEC, University of Lausanne

Companion deck to `08_Heterogeneous_Agents_Youngs_Method`.

Based on: Azinovic-Yang & Žemlička (2025), Boppart et al. (2018), Auclert et al. (2021).

Motivation

- Globally solving models with rich heterogeneity and aggregate risk is hard.
  - $\rightarrow$ high-dimensional endogenous aggregate state (a distribution μ_t).
- Sequence-space idea: approximate equilibrium objects as a function of a truncated history of aggregate shocks.

$$\underbrace{\mathcal{V}\big(a_t,\, z_t,\, \mu_t,\, \varepsilon_t\big)}_{\text{state-space form}} = \underbrace{\mathcal{V}\big(a_t,\, z_t,\, \{\varepsilon_i\}_{i=0}^{t}\big)}_{\text{sequence-space form}} \approx \underbrace{\mathcal{V}\big(a_t,\, z_t,\, \{\varepsilon_i\}_{i=t-\tau}^{t}\big)}_{\text{truncated sequence form}}$$

- Truncated history is high-dimensional, but exogenous.
- Can deep learning approximate equilibrium objects on this exogenous domain?

Roadmap of This Lecture

Today: the Brock–Mirman warm-up

The simplest setting in which the history-to-policy idea is visible. One representative household, one aggregate shock, an analytic policy under $\delta = 1$, log utility for cross-checking.

Two paper-level ingredients (Azinovic-Yang & Žemlička, 2025)

1. Approximate equilibrium objects as a function of truncated histories – exogenous domain. **(covered today)**
2. Parameterise via shape-preserving neural networks – monotonicity / concavity guarantees. **(pointer only)**

Notebooks for this lecture

`05_SequenceSpace_BrockMirman.ipynb` – the warm-up that isolates the history-to-policy logic.
`05b_SequenceSpace_IRBC.ipynb` – bridge to the IRBC of Day 3 in sequence space.

Heterogeneous-agent application – self-study only

Companion deck `08_Heterogeneous_Agents_Youngs_Method.pdf` (heterogeneous agents = self-study); reference notebook `06_SequenceSpace_KrusellSmith.ipynb` and the JAX implementation `KrusellSmith_Tutorial_CPU.ipynb`.

Learning objectives

By the end of this lecture you should be able to:

- Reformulate a state-space equilibrium as a truncated sequence-space equilibrium, mapping endogenous state into exogenous shock history
- Diagnose ergodicity and justify truncation length using decay rates of partial derivatives
- Implement residual minimization to train a neural-network policy on exogenous shock histories
- Compare state-space and sequence-space deep learning and recognize the stability advantages of exogenous inputs
- Build and validate a history-dependent policy network using the cloud method on Monte Carlo shock paths

Method Overview

State-Space Formulation of Model Equilibria

- **Endogenous variables** y_t are a function of the state vector x_t:

$$y_t = \underbrace{f}_{\substack{\text{recursive}\\ \text{solution}}}(x_t).$$

- **Transition function** determines next-period states:

$$x_{t+1} = \underbrace{H}_{\substack{\text{known}\\ \text{function}}}(x_t,\, y_t,\, \varepsilon_{t+1}).$$

- **Equilibrium:** f satisfies a system of equilibrium conditions

$$\boxed{\underbrace{G(f,x) = 0}_{\text{functional equation}} \qquad \forall x\,.}$$

Domain of f: state space $\mathcal{X}$. Once μ_t enters x_t, this is high-dimensional.

Sequence-Space Formulation of Model Equilibria

- **Endogenous variables** are a function of the history of shock realisations:

$$y_t \;=\; \underbrace{\Psi}_{\substack{\text{seq. space}\\ \text{solution}}} \big(\underbrace{\varepsilon_t,\, \varepsilon_{t-1},\, \varepsilon_{t-2},\, \ldots}_{\text{history}} \;\big|\; x_0\big).$$

- **State** is then a function of the history and the initial condition:

$$x_t \;=\; \underbrace{\mathcal{H}}_{\substack{\text{iterated}\\ \text{law of motion}}} \big(\mathcal{E}_t,\, x_0 \;\big|\; \Psi\big).$$

- **Equilibrium:** Ψ satisfies an equivalent set of conditions

$$\boxed{G(\Psi, \mathcal{E}, x_0) \;=\; 0 \quad \forall \mathcal{E},\ \forall x_0\ .}$$

Domain of Ψ: shock history $\mathcal{E}_t = (\varepsilon_t, \varepsilon_{t-1}, \ldots)$ – exogenously driven.

Ergodicity and Truncation

- A large class of economies feature ergodic dynamics:

$$\lim_{\tau\to\infty} \frac{\partial \Psi}{\partial \varepsilon_{t-\tau}} = 0.$$

  The influence of past shocks decays. For sufficiently high τ, truncation error can be made arbitrarily low.

- We search for a truncated sequence-space solution:

$$y_t \approx \underbrace{\widehat{\Psi}}_{\substack{\text{trunc. seq.}\\ \text{space sol.}}} \underbrace{\left(\varepsilon_t, \varepsilon_{t-1}, \ldots, \varepsilon_{t-\tau}\right)}_{\text{truncated history}}.$$

- **Brock–Mirman makes it concrete** ($\delta = 1$, log utility, $s = \alpha\beta$):

$$\log K_t = c + \log A_{t-1} + \alpha \log A_{t-2} + \alpha^2 \log A_{t-3} + \cdots$$

  Lag-τ weight is $\alpha^{\tau-1}$ (with A_{t-1} at lag 1). At $\tau = 25$, $\alpha = 1/3$, the weight $\alpha^{24} \approx 3.5 \times 10^{-12}$.

Deep Learning in the Sequence Space

- **Challenge:** $\widehat{\Psi}$ is a high-dimensional object.
- **Solution:** deep learning.
  1. Parameterise $\widehat{\Psi}$ using a neural network $\mathcal{N}_\theta(\mathcal{E})$.
  2. Train the network, i.e. find $\underbrace{\theta}_{\substack{\text{netw.}\\ \text{param.}}}$ such that $\mathcal{N}_\theta(\mathcal{E}) \approx \widehat{\Psi}$.
- **Two algorithms:**
  1. Direct residual minimisation: define residual by inserting $\mathcal{N}_\theta$ into both sides of the Euler equation; SGD on mean squared residual.
  2. Time iteration with EGM: more involved, more flexible – handles non-convex choices and multiple roots.

Today: residual minimisation. Time iteration is in the appendix.

Algorithm Sketch: Residual Minimisation

1. **Initialisation:** set iter $= 1$.
   - initialise network parameters θ^0;
   - sample initial state-history pairs $\left\{(x_j^0, \mathcal{E}_j^0)\right\}_{j=1}^N$.

Algorithm Sketch: Residual Minimisation

1. **Initialisation:** set iter $= 1$.
   - initialise network parameters θ^0;
   - sample initial state-history pairs $\left\{(x_j^0, \mathcal{E}_j^0)\right\}_{j=1}^N$.
2. **Training loop:**

   2.1 Simulation. Fixing θ^{iter}, sample shocks and simulate the policies T periods forward.
   - start with $\left\{(x_j^{\text{iter}-1}, \mathcal{E}_j^{\text{iter}-1})\right\}_{j=1}^N$;
   - collect terminal pairs $\left\{(x_j^{\text{iter}}, \mathcal{E}_j^{\text{iter}})\right\}_{j=1}^N$.

Algorithm Sketch: Residual Minimisation

1. **Initialisation:** set iter $= 1$.
   - initialise network parameters θ^0;
   - sample initial state-history pairs $\left\{(x_j^0, \mathcal{E}_j^0)\right\}_{j=1}^N$.
2. **Training loop:**

   2.1 Simulation. Fixing θ^{iter}, sample shocks and simulate the policies T periods forward.
   - start with $\left\{(x_j^{\text{iter}-1}, \mathcal{E}_j^{\text{iter}-1})\right\}_{j=1}^N$;
   - collect terminal pairs $\left\{(x_j^{\text{iter}}, \mathcal{E}_j^{\text{iter}})\right\}_{j=1}^N$.

   2.2 Policy improvement. Evaluate the mean squared residual and its gradient.
   - for each pair j: evaluate θ^{iter} to get current endogenous y_j;

Algorithm Sketch: Residual Minimisation

1. **Initialisation:** set iter $= 1$.
   - initialise network parameters θ^0;
   - sample initial state-history pairs $\left\{(x_j^0, \mathcal{E}_j^0)\right\}_{j=1}^N$.
2. **Training loop:**

   2.1 Simulation. Fixing θ^{iter}, sample shocks and simulate the policies T periods forward.
   - start with $\left\{(x_j^{\text{iter}-1}, \mathcal{E}_j^{\text{iter}-1})\right\}_{j=1}^N$;
   - collect terminal pairs $\left\{(x_j^{\text{iter}}, \mathcal{E}_j^{\text{iter}})\right\}_{j=1}^N$.

   2.2 Policy improvement. Evaluate the mean squared residual and its gradient.
   - for each pair j: evaluate θ^{iter} to get current endogenous y_j;
   - use y_j and M-point quadrature to construct $\left\{(x'_{j,m}, \mathcal{E}'_{j,m})\right\}_{m=1}^M$;

Algorithm Sketch: Residual Minimisation

1. **Initialisation:** set iter $= 1$.
   - initialise network parameters θ^0;
   - sample initial state-history pairs $\left\{(x_j^0, \mathcal{E}_j^0)\right\}_{j=1}^N$.
2. **Training loop:**

   2.1 Simulation. Fixing θ^{iter}, sample shocks and simulate the policies T periods forward.
   - start with $\left\{(x_j^{\text{iter}-1}, \mathcal{E}_j^{\text{iter}-1})\right\}_{j=1}^N$;
   - collect terminal pairs $\left\{(x_j^{\text{iter}}, \mathcal{E}_j^{\text{iter}})\right\}_{j=1}^N$.

   2.2 Policy improvement. Evaluate the mean squared residual and its gradient.
   - for each pair j: evaluate θ^{iter} to get current endogenous y_j;
   - use y_j and M-point quadrature to construct $\left\{(x'_{j,m}, \mathcal{E}'_{j,m})\right\}_{m=1}^M$;
   - evaluate the policy at quadrature nodes; compute $\mathbb{E}[\cdots]$ and the equilibrium-conditions residual $\mathcal{R}_j$;

Algorithm Sketch: Residual Minimisation

1. **Initialisation:** set iter $= 1$.
   - initialise network parameters θ^0;
   - sample initial state-history pairs $\left\{(x_j^0, \mathcal{E}_j^0)\right\}_{j=1}^N$.
2. **Training loop:**

   2.1 Simulation. Fixing θ^{iter}, sample shocks and simulate the policies T periods forward.
   - start with $\left\{(x_j^{\text{iter}-1}, \mathcal{E}_j^{\text{iter}-1})\right\}_{j=1}^N$;
   - collect terminal pairs $\left\{(x_j^{\text{iter}}, \mathcal{E}_j^{\text{iter}})\right\}_{j=1}^N$.

   2.2 Policy improvement. Evaluate the mean squared residual and its gradient.
   - for each pair j: evaluate θ^{iter} to get current endogenous y_j;
   - use y_j and M-point quadrature to construct $\left\{(x'_{j,m}, \mathcal{E}'_{j,m})\right\}_{m=1}^M$;
   - evaluate the policy at quadrature nodes; compute $\mathbb{E}[\cdots]$ and the equilibrium-conditions residual $\mathcal{R}_j$;
   - compute $\frac{1}{N}\sum_j \mathcal{R}_j^2$, get its gradient via backprop, update θ by SGD.

Algorithm Sketch: Residual Minimisation

1. **Initialisation:** set iter $= 1$.
   - initialise network parameters θ^0;
   - sample initial state-history pairs $\left\{(x_j^0, \mathcal{E}_j^0)\right\}_{j=1}^N$.
2. **Training loop:**

   2.1 Simulation. Fixing θ^{iter}, sample shocks and simulate the policies T periods forward.
   - start with $\left\{(x_j^{\text{iter}-1}, \mathcal{E}_j^{\text{iter}-1})\right\}_{j=1}^N$;
   - collect terminal pairs $\left\{(x_j^{\text{iter}}, \mathcal{E}_j^{\text{iter}})\right\}_{j=1}^N$.

   2.2 Policy improvement. Evaluate the mean squared residual and its gradient.
   - for each pair j: evaluate θ^{iter} to get current endogenous y_j;
   - use y_j and M-point quadrature to construct $\left\{(x'_{j,m}, \mathcal{E}'_{j,m})\right\}_{m=1}^M$;
   - evaluate the policy at quadrature nodes; compute $\mathbb{E}[\cdots]$ and the equilibrium-conditions residual $\mathcal{R}_j$;
   - compute $\frac{1}{N}\sum_j \mathcal{R}_j^2$, get its gradient via backprop, update θ by SGD.

   2.3 Check. If loss is sufficiently low, stop; else iter $+= 1$ and return to (i).

Why State-Space DL Is Fragile – and How Sequence Space Helps

Failure mode of state-space DL in HA models

1. Network update $\theta^{(k)} \to \theta^{(k+1)}$ shifts the policy.
2. Forward simulation under the new policy shifts the distribution $\{\mu_t\}$ that feeds the network.
3. Network is now evaluated on previously unseen aggregate states.
4. Out-of-distribution predictions $\Rightarrow$ large residual $\Rightarrow$ large gradient.
5. Large gradient $\Rightarrow$ large policy update. Return to step 1.

Sequence space breaks the loop. The network input is the truncated shock history $\mathcal{E}_t$, drawn from the exogenous aggregate-shock law of motion. That distribution is fixed by model primitives and does not move with the policy update.

The single biggest source of stability gains documented in Azinovic-Yang & Žemlička (2025).

Brock–Mirman: What Exactly Changes?

State-space warm-up (discrete-time DEQN warm-up)

- **Input:** (K_t, A_t) – 2 floats.
- **Network output:** C_t (or savings rate).
- **Transition:** $K_{t+1} = A_t K_t^\alpha + (1-\delta)K_t - C_t$.

Sequence-space warm-up (this notebook)

- **Input:** $\mathcal{E}_t^T = (\varepsilon_{t-T+1}, \ldots, \varepsilon_t)$ – 25 floats.
- **Network output:** savings rate s.
- **Recover:** $K_{t+1} = s \cdot y_t$ with $y_t = A_t K_t^\alpha + (1-\delta)K_t$.

Main point

The economic model and Euler equation do not change. What changes is the computational representation: the network sees a shock history rather than the current endogenous state.

Note: in BM the input swap is larger, not smaller – sequence space here is a teaching device, not a dimensionality reduction. The payoff appears in the HA setting (next slide).

Hands-On Notebook: 05_SequenceSpace_BrockMirman.ipynb

Cell-by-cell map

1. §1–4 markdown: model, sequence-space formulation, ergodicity, algorithm sketch.
2. §5–8 setup: imports, calibration ($\alpha = 1/3$, $\beta = 0.97$, $\gamma = \delta = 1$), Gauss–Hermite (8 nodes), steady-state init.
3. §11 network: 25-input, 3×64 GELU, sigmoid output – savings rate.
4. §12–13 loss core: `Policy_sequence`, then `E_sequence` (Euler residual; network called twice – at $\mathcal{E}_t$ and at every quadrature node $\mathcal{E}_{t+1}$).
5. §14–16 loop: forward simulator (history rolls one period), train step, cloud-method outer loop.
6. §17 diagnostics: loss curve, ergodic (K, z) cloud, BM closed-form comparison.

What to expect

$\sim$20–40 min on CPU at the defaults. Loss reaches $\sim 10^{-6}$; trained savings policy overlays the analytic $\alpha\beta\, y$ on a 32-point sample.

Where Sequence Space Actually Pays Off – HA in One Slide

Smaller, exogenous network input

- Histogram DEQN feeds a ∼100-bin wealth histogram into the network.
- Sequence-space DEQN feeds a shock history of ∼25 floats.
- Histogram input moves with the policy; shock history is fixed by primitives.

Operator-learning + shape preservation

- Network outputs policy coefficients, not a single decision.
- Non-negative I-spline coefficients $\Rightarrow$ MPC monotone, c concave, $c \leq m$ by construction, no shape penalties.
- Young's method still propagates μ inside the simulator, exact aggregates.

Where to look

Companion deck `08_Heterogeneous_Agents_Youngs_Method.pdf`; teaching notebook `06_SequenceSpace_KrusellSmith.ipynb` (TF2); JAX reference `KrusellSmith_Tutorial_CPU.ipynb`.

Summary

Key messages

1. Sequence space changes the network input, not the equilibrium conditions.
2. Ergodicity-and-truncation: long histories of exogenous shocks summarize the relevant economic state, with truncation error decaying geometrically.
3. Brock–Mirman is the clean warm-up; HA models are where the method genuinely pays off (smaller, exogenous input + breaks the state-space feedback loop).
4. Two algorithms: residual minimisation (today) and time iteration with EGM (appendix).

References

- Azinovic-Yang, M., & Žemlička, J. (2025). *Deep learning in the sequence space.* arXiv:2509.13623.
- Auclert, A., Bardóczy, B., Rognlie, M., & Straub, L. (2021). Using the sequence-space Jacobian to solve and estimate heterogeneous-agent models. *Econometrica* 89(5), 2375–2408.
- Boppart, T., Krusell, P., & Mitman, K. (2018). Exploiting MIT shocks in heterogeneous-agent economies. *Journal of Economic Dynamics & Control* 89, 68–92.
- Carroll, C.D. (2006). The method of endogenous gridpoints for solving dynamic stochastic optimization problems. *Economics Letters* 91(3), 312–320.
- Companion code repository: `https://github.com/azinoma/DeepLearningInTheSequenceSpace`.

Appendix: Backup Slides

Krusell–Smith implementation details, alternative algorithm, and practical heuristics.

KS Tutorial: The Economy in One Slide

Households (continuum)

- Choose c_t to maximize $\mathbb{E}_0 \sum_{t \geq 0} \beta^t \log c_t$.
- Budget: $c_t + k_{t+1} = w_t \, \varepsilon_t + R_t \, k_t$, $k_{t+1} \geq 0$.
- Idiosyncratic efficiency $\varepsilon_t \in \{\varepsilon_L, \varepsilon_H\}$, 2-state Markov.

Firm (Cobb–Douglas)

- $Y = ZK^{\alpha} L^{1-\alpha}$.
- $R_t = 1 - \delta + \alpha Z_t (K_t / L_t)^{\alpha - 1}$.
- $w_t = (1 - \alpha) Z_t (K_t / L_t)^{\alpha}$.

Aggregate shock

$Z_t \in \{Z_L, Z_H\}$, 2-state Markov.

Calibration in the notebook

α	0.36
δ	0.025
β	0.93
ε values	$\{0.5, 1.5\}$
Z values	$\{0.93, 1.07\}$
Π_ε persistence	0.9
Π_Z persistence	0.7

KS Tutorial: I-Spline MPC Parameterisation

What the network outputs (per ε)

- Boundary MPC $\alpha(\varepsilon) \in (0,1)$ via sigmoid.
- Non-negative weights $\tilde{w}_j(\varepsilon) \geq 0$ with $\sum_j \tilde{w}_j < 1$ (phantom-zero softmax).

Grid MPC (decreasing by construction)

$$\mathrm{MPC}_{\varepsilon,n} \;=\; \alpha(\varepsilon)\Big(1 - \sum_{j=1}^{J} \tilde{w}_j(\varepsilon)\, B_{j,n}\Big)$$

with $B_{j,n} = I_j(\log(\text{shift} + k_n))$.

From MPC to consumption

$$c(k_0) = \mathrm{MPC}(k_0) \cdot m(k_0)$$

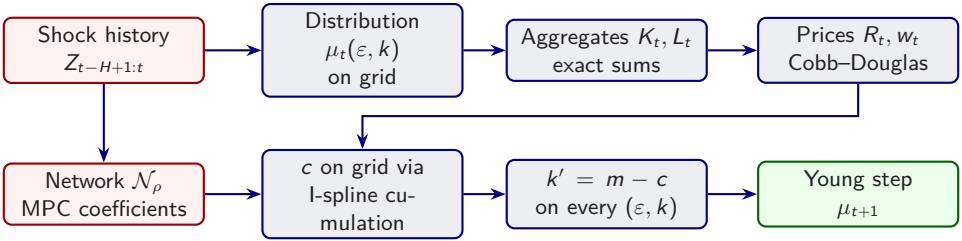
$$c(k_n) = c(k_{n-1}) + \mathrm{MPC}(k_n)\, R\, \Delta k_n$$

with cash-on-hand $m = w\varepsilon + Rk$.

Shape guarantees by construction

c increasing, c concave, $c \leq m$, $\mathrm{MPC} \in [0,1]$. No shape penalties needed in the loss.

KS Tutorial: One Period in the Simulator



Why Young inside the simulator

Aggregates $K = \sum k\,\mu(\varepsilon, k)$ and $L = \sum \varepsilon\,\mu$ are exact weighted sums on the grid. No Monte-Carlo noise floor on prices.

KS Tutorial: Fischer–Burmeister KKT Loss

Two regimes per household

- Interior ($k' > 0$): Euler holds with equality, $g = 0$.
- Constrained ($k' = 0$): Euler gap $g \geq 0$.

Residuals on the grid

$$g = \frac{c_{\text{Euler}} - c}{c}, \qquad s = \frac{k'}{c}.$$

With $c_{\text{Euler}} = 1/q$ and

$$q = \beta\, \mathbb{E}_t\big[\, R'\, u'(c')\big]$$

evaluated by integrating across (ε', Z').

Fischer–Burmeister envelope

$$\mathrm{FB}(g, s) = \sqrt{g^2 + s^2 + \epsilon_{\text{fb}}} - g - s.$$

$\mathrm{FB} = 0$ iff KKT holds.

Loss

$$\mathcal{L}(\rho) = \mathbb{E}_{\text{buffer}}\Big[\overline{\mathrm{FB}^2}\Big]$$

mean over the (ε, k) grid, averaged over replay states.

KS Tutorial: Replay Buffer and Training Loop

Buffer state = pair (Z-history, μ).

Each epoch

1. Draw N_{agg} buffer entries; new shock paths.
2. Roll forward T periods with current $\mathcal{N}_\rho$ to update μ via Young (`stop_gradient`).
3. Append terminal (Z-hist, μ) pairs, evict oldest.
4. Take `FB_STEPS` mini-batch SGD steps on FB^2.

Notebook defaults

History length H	50
Roll-forward T	100
Buffer cap	128
N_{agg} per epoch	8
FB steps per epoch	10
FB mini-batch	16
Optimiser	Adam, 5×10^{-5}
Grad clip	$\lVert \cdot \rVert_2 \leq 1$

Two Training Algorithms: Residual Min vs. Time Iteration

Direct residual minimisation

- Train on $\mathbb{E}[\mathrm{FB}^2]$ via SGD.
- One loss, one optimiser.
- Best for smooth, single-root Euler equations.

Time iteration with EGM

(Carroll, 2006; Azinovic-Yang & Žemlička, 2025)

- Each outer step: use $\mathcal{N}_\rho$ for next-period policy, EGM-invert to get current targets, regress.
- Per-batch root-finding step.
- Handles non-convex choices, multiple roots, ZLB, non-monotone Laffer.

When to switch

Use residual minimisation by default. Switch to time iteration when convergence stalls, when the Euler equation has multiple roots, or when discrete choices appear in the optimization.

KS Tutorial: What the Diagnostics Show

Convergence

- $\log \mathrm{FB}^2$ vs. epoch.
- Target accuracy $\sim 10^{-6}$ on the simplified KS economy.

Learned policy

- $c(k)$ and MPC plotted at all-low and all-high Z histories.
- Differences across Z histories reveal how the network reads aggregate state through the shock window.

Approximate aggregation scatter

- Plot $c(\varepsilon, k)$ and MPC at fixed k against K across replay states.
- Tight clouds = the policy depends mainly on contemporaneous prices.

Euler/FB error bands

- Median and 10–90% percentile bands of $|\mathrm{FB}|$ across the buffer.
- Higher near the constraint $k \approx 0$ and at high k with little household mass.

Practical Heuristics

Truncation length T – three rules

1. **OLG:** two life-cycles (covers all shocks experienced by any living household).
2. **Iterative:** start short, increase until residual stops dropping.
3. **Overkill:** $\rho^T_{\text{shock}} \leq 10^{-8}$. Long histories are cheap to feed.

Innovations or realisations?

For AR(1) shocks, feed realisations z_t^T rather than innovations ε_t^T. Levels integrate the persistence, so shorter T suffices. Feeding both is most accurate.

What to approximate

Smooth, bounded objects (MPCs, savings rates) yield much better Euler accuracy than raw levels (consumption, savings). This is why I-spline MPCs work so well.

When the method breaks

Truncation relies on *ergodicity*: $\partial\Psi/\partial\varepsilon_{t-\tau} \to 0$ as $\tau \to \infty$. Limit cycles or deterministic chaos violate this; convergence is then not guaranteed.